



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной
и дополненной реальности»

Практическая работа № 4

Студент группы *ИКБО-50-23, Павлов Н.С.*

(подпись)

Старший
преподаватель *Горбатов Г.В.*

(подпись)

Отчет представлен «__» _____ 2026 г.

Москва 2026 г.

1. Система частиц была использована в проекте для реализации эффектов появления кубов и получения урона (рис. 1 – 2).

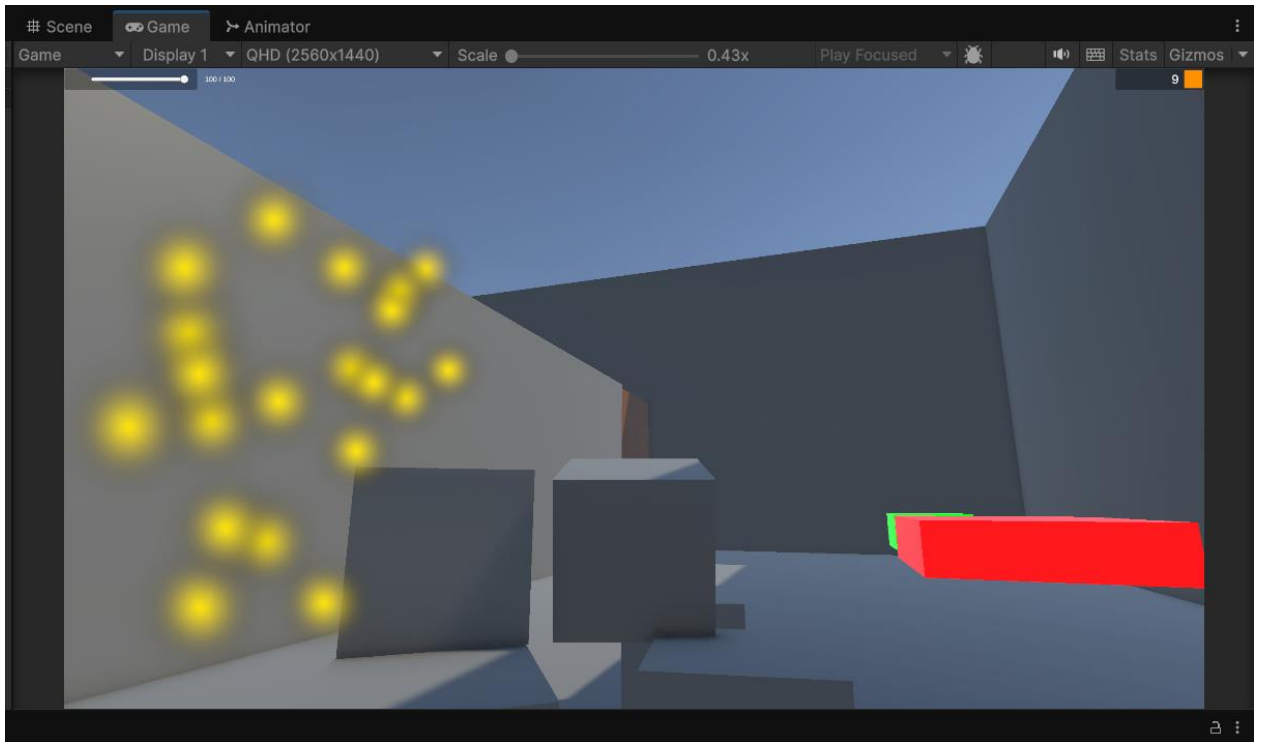


Рисунок 1 — Эффект появления кубов

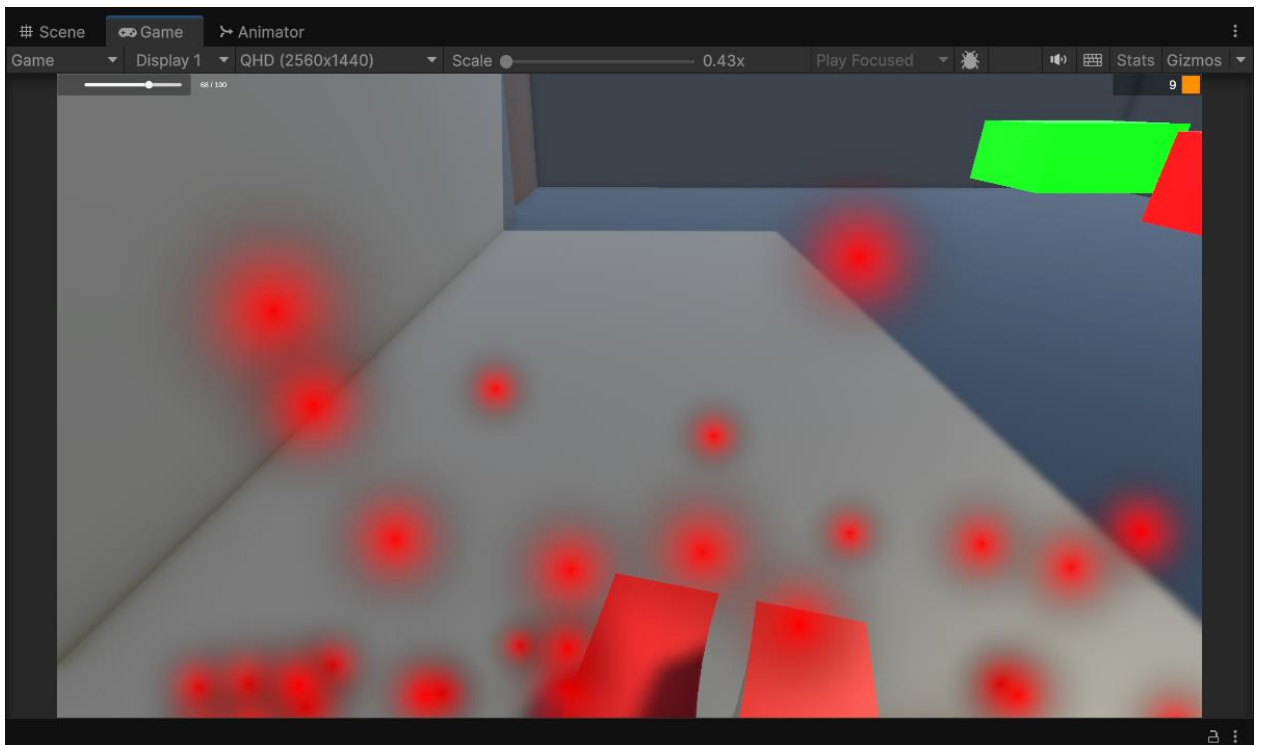


Рисунок 2 — Эффект получения урона

Пример с применением частиц в коде показан в листинге скрипта

получения урона ниже.

Листинг 1 - Код скрипта DamageEffect.cs на C#

```
using UnityEngine;
using System.Collections;

public class DamageEffect : MonoBehaviour
{
    [Header("Визуальные эффекты")]
    [SerializeField] private ParticleSystem damageParticles;
    [SerializeField] private float flashDuration = 0.1f;

    private Renderer playerRenderer;
    private Color originalColor;
    private AudioSource audioSource;

    private void Start()
    {
        playerRenderer = GetComponent<Renderer>();
        if (playerRenderer != null)
        {
            originalColor = playerRenderer.material.color;
        }
    }

    public void PlayDamageEffect()
    {
        // Частицы
        if (damageParticles != null)
        {
            ParticleSystem effect = Instantiate(damageParticles, transform.position, Quaternion.identity);
            effect.Play();
            Destroy(effect.gameObject, 1f);
        }

        // Красная вспышка
        if (playerRenderer != null)
        {
            StartCoroutine(FlashRed());
        }
    }

    private IEnumerator FlashRed()
    {
        playerRenderer.material.color = Color.red;
        yield return new WaitForSeconds(flashDuration);
        playerRenderer.material.color = originalColor;
    }
}
```

2. Для плавной и живой картинки использовалась анимация двух видов. Для вращения пропеллера у дрона-противника использовалась процедурная анимация (листинг 2).

Листинг 2 - Код скрипта *PropellerSpin.cs* на C#

```
using UnityEngine;

public class PropellerSpin : MonoBehaviour
{
    [Header("Настройки вращения")]
    [SerializeField] private float rotationSpeed = 500f;
    [SerializeField] private Vector3 rotationAxis = Vector3.up;

    private void Update()
    {
        // Вращаем пропеллер вокруг заданной оси
        transform.Rotate(rotationAxis * rotationSpeed * Time.deltaTime);
    }
}
```

Для анимации открытия и закрытия двери был использован Animator.

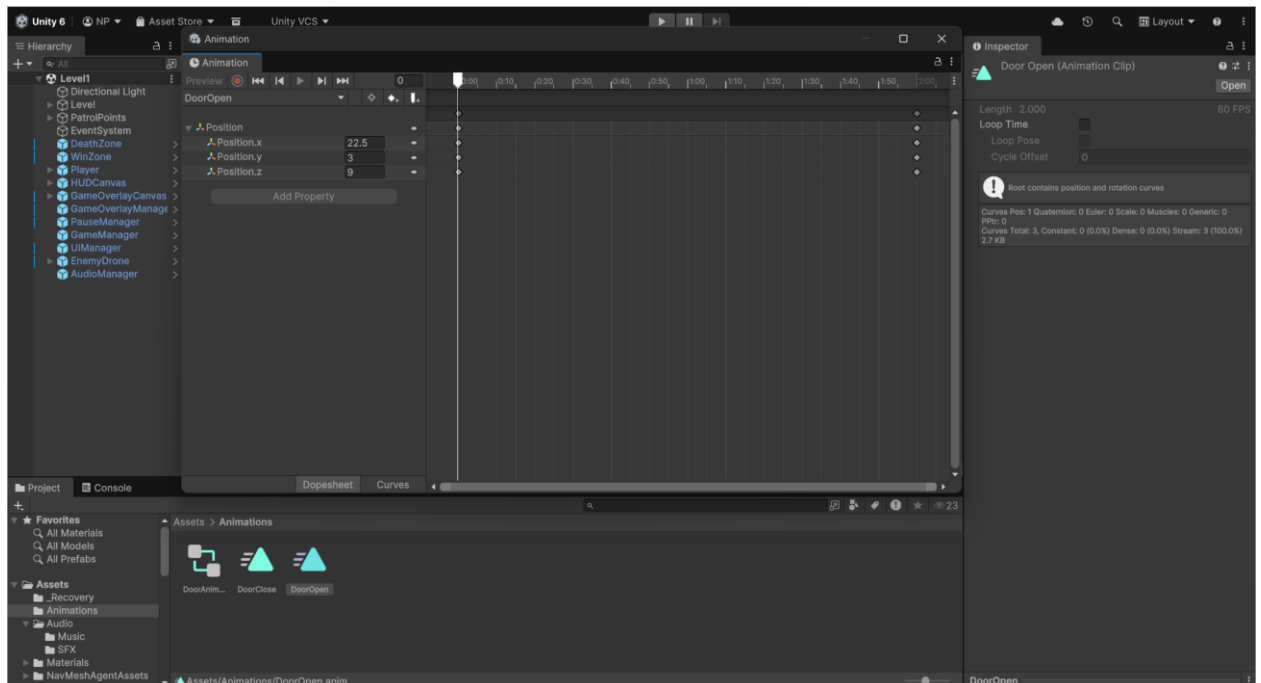


Рисунок 3 — Покадровая настройка анимации открытия двери

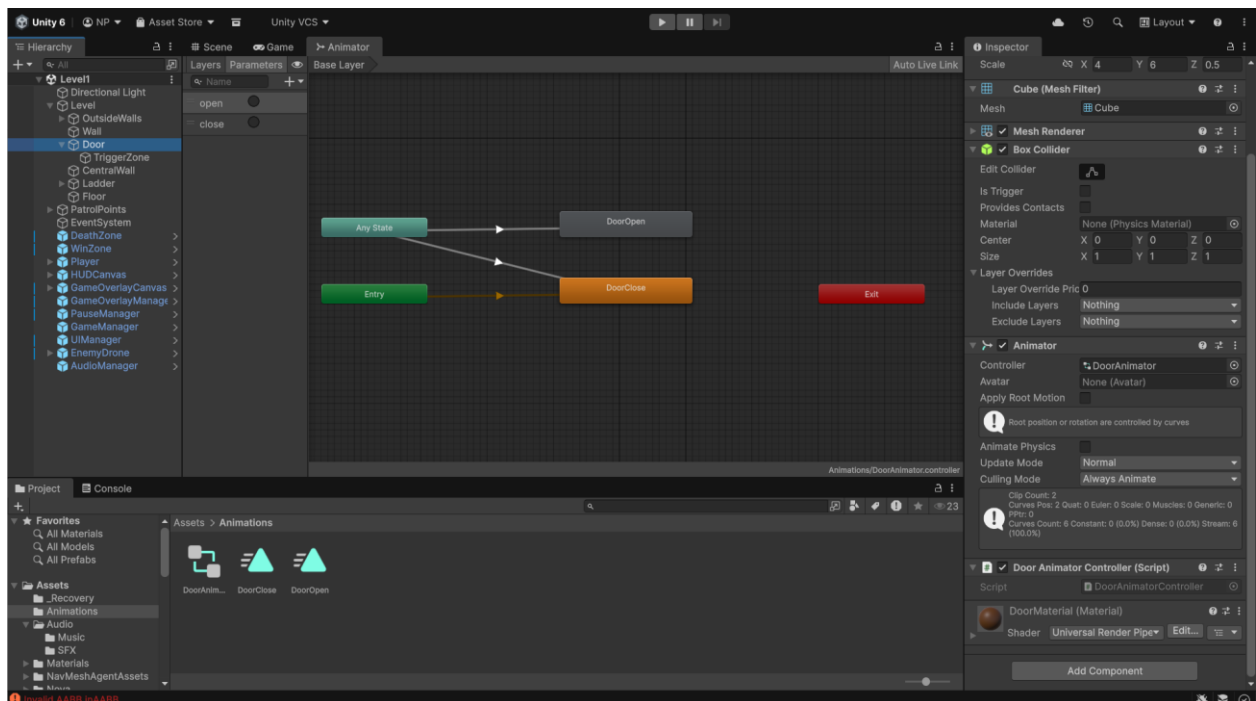


Рисунок 4 — Параметры Animator

Листинг 3 - Код скрипта DoorTrigger.cs на C#

```
using UnityEngine;

public class DoorTrigger : MonoBehaviour
{
    [SerializeField] private DoorAnimatorController door;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
            door.OpenDoor();
    }

    private void OnTriggerExit(Collider other)
    {
        if (other.CompareTag("Player"))
            door.CloseDoor();
    }
}
```

Листинг 4 - Код скрипта DoorAnimatorController.cs на C#

```
using UnityEngine;

public class DoorAnimatorController : MonoBehaviour
{
    private Animator doorAnimator;
    private bool isOpen = false;
```

```

private void Start()
{
    doorAnimator = GetComponent<Animator>();
}

public void OpenDoor()
{
    if (isOpen) return;
    isOpen = true;
    doorAnimator.SetTrigger("open");
    AudioManager.Instance?.PlayDoorOpen();
}

public void CloseDoor()
{
    if (!isOpen) return;
    isOpen = false;
    doorAnimator.SetTrigger("close");
    AudioManager.Instance?.PlayDoorClose();
}
}

```

3. Для еще более живой и комфортной игры были добавлены звуки и фоновая музыка. На все сцены был добавлен менеджер управления звуками. Менеджер реализует скрипт с управлением звуками, на который ссылаются методы всех других скриптов. С помощью него реализован центральный контроллер и микшер музыки и звуков.



Рисунок 5 — Менеджер управления звуками

Листинг 5 - Код скрипта *AudioManager.cs* на C#

```
using UnityEngine;

public class AudioManager : MonoBehaviour
{
    public static AudioManager Instance;
```

```

[Header("Звуки игрока")]
[SerializeField] private AudioClip jumpSound;
[SerializeField] private AudioClip footstepSound;
[SerializeField] private AudioClip damageSound;

[Header("Звуки игры")]
[SerializeField] private AudioClip cubeSpawnSound;
[SerializeField] private AudioClip winSound;
[SerializeField] private AudioClip loseSound;
[SerializeField] private AudioClip buttonClickSound;

[Header("Звуки врага")]
[SerializeField] private AudioClip enemyDetectSound;

[Header("Звуки окружения")]
[SerializeField] private AudioClip doorOpenSound;
[SerializeField] private AudioClip doorCloseSound;

[Header("Фоновая музыка")]
[SerializeField] private AudioClip backgroundMusic;

[Header("Настройки")]
[SerializeField] private float musicVolume = 0.3f;
[SerializeField] private float sfxVolume = 0.7f;

private AudioSource musicSource;
private AudioSource sfxSource;

private float lastFootstepTime = 0f;
private float lastDamageTime = 0f;
private readonly float footstepCooldown = 0.4f;
private readonly float damageCooldown = 0.5f;

private void Awake()
{
    if (Instance == null)
    {
        Instance = this;
        DontDestroyOnLoad(gameObject);

        musicSource = gameObject.AddComponent<AudioSource>();
        sfxSource = gameObject.AddComponent<AudioSource>();

        musicSource.loop = true;
        musicSource.volume = musicVolume;

        sfxSource.volume = sfxVolume;
    }
    else
    {
        Destroy(gameObject);
    }
}

```



```

    }
}

private void Start()
{
    if (backgroundMusic != null)
        PlayMusic(backgroundMusic);
}

public void PlayJump() => PlaySFX(jumpSound, 0.6f);

public void PlayFootstep()
{
    if (Time.time - lastFootstepTime < footstepCooldown) return;
    lastFootstepTime = Time.time;
    PlaySFX(footstepSound, 0.4f);
}

public void PlayDamage()
{
    if (Time.time - lastDamageTime < damageCooldown) return;
    lastDamageTime = Time.time;
    PlaySFX(damageSound, 0.8f);
}

public void PlayCubeSpawn() => PlaySFX(cubeSpawnSound, 0.7f);
public void PlayWin() => PlaySFX(winSound, 0.8f);
public void PlayLose() => PlaySFX(loseSound, 0.8f);
public void PlayButtonClick() => PlaySFX(buttonClickSound, 0.6f);
public void PlayEnemyDetect() => PlaySFX(enemyDetectSound, 0.7f);
public void PlayDoorOpen() => PlaySFX(doorOpenSound, 0.5f);
public void PlayDoorClose() => PlaySFX(doorCloseSound, 0.5f);

public void PlayMusic(AudioClip clip)
{
    if (clip == null) return;
    if (musicSource.clip == clip && musicSource.isPlaying) return;

    musicSource.clip = clip;
    musicSource.Play();
}

public void StopMusic() => musicSource.Stop();

private void PlaySFX(AudioClip clip, float volumeMultiplier = 1f)
{
    if (clip != null)
        sfxSource.PlayOneShot(clip, sfxVolume * volumeMultiplier);
}
}

```

